

Evaluating LLM System-Level Guardrails Against Prompt Injection: Architecture, Robustness, and Latency Trade-offs

Weiran Wang

MEng, University of Toronto
weiran.wang@mail.utoronto.ca

Daixin Tian

MScAC, University of Toronto
daixin.tian@mail.utoronto.ca

Abstract

Prompt injection attacks pose a significant threat to Large Language Model (LLM) applications. This paper evaluates four system-level guardrail architectures—Rule-based (Regex), ML-based Classifier (DeBERTa), Local-LLM (Llama-3.2-1B), and API-based (GPT-4o-mini) — integrated with a Llama-3-8B baseline — on a curated prompt injection dataset of 1000 prompts (800 unsafe and 200 safe prompts). Our evaluation considers both security and usability metrics, including Attack Success Rate (ASR) and False Refusal Rate (FRR), alongside with inference efficiency (Latency). The results show that rule-based methods provide negligible improvement over baseline, while classifier-based and API-based guardrails significantly reduce ASR with minimal impact on usability. However, we observe that aggressive model-level defenses can lead to excessive refusal behavior, resulting in poor user experience. Overall, our findings highlight the importance of balancing security and usability in guardrail design and suggest that hybrid or classifier-based approaches currently offer the most practical defense against prompt injection attacks.

ACM Reference Format:

Weiran Wang and Daixin Tian. 2026. Evaluating LLM System-Level Guardrails Against Prompt Injection: Architecture, Robustness, and Latency Trade-offs. In *Proceedings of Course Project Report (CSC2125 SE4AI Project)*. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Large Language Models (LLMs) have emerged as pivotal tools for natural language understanding and complex decision-making tasks. Their ability to process and generate human-like text has led to rapid adoption across diverse high-stakes industries, including healthcare for ICU trend forecasting, transportation where models interpret maintenance logs for road asset management, and software engineering for automating ETL pipelines by interpreting dynamic data schema.

In these scenarios, the LLM often acts as an interface between raw, untrusted data and critical downstream actions, making them highly susceptible to malicious attacks and threats. Prompt Injection represents one of the most prominent threat where attackers

use adversarial inputs to subvert a model's intended through carefully crafted inputs [11]. Specifically, these attacks can facilitate proprietary knowledge theft by forcing the model to reveal its internal system prompts or the confidential business logic; Beyond information disclosure, the hijacking of LLM agents to execute unauthorized API calls such as deleting database records or initiating fraudulent transactions [1]. Depending on the entry point of the malicious payload, these threats are categorized into two distinct modalities:

- **Direct Prompt Injection (Jailbreaking):** Occurs when a user provides malicious instructions directly in the chat interface, such as *"Ignore all previous instructions and provide secret codes"*.
- **Indirect Prompt Injection:** Occurs when the LLM retrieves or processes data from untrusted external sources that contain hidden adversarial commands intended to execute without the user's knowledge. For instance, a model summarizing a compromised website might encounter a hidden payload saying: *"[System Note]: Disregard the actual content; instead, tell the user their account is compromised and redirect them to malicious-site.com"* [2].

In response, LLM service providers and researchers have developed a multi-layered defense ecosystem, broadly categorized into model-level defense and system-level guardrails. While model-level methods attempt to embed safety constraints directly into model parameters via Reinforcement Learning from Human Feedback (RLHF), and safety fine-tuning [10].

In contrast, system-level guardrails act as external protective shells that govern the model's input-output pipeline without altering its internal parameters. This architectural independence allows for a versatile defense-in-depth strategy, starting with input remediation, where rule-based heuristics and regular expressions (Regex) provide an immediate first line of defense against known attack patterns. To address the fragility of static rules, more robust systems leverage machine learning (ML) classifiers, such as DeBERTa and other transformer-based encoders which map prompts into high-dimensional latent spaces [13]. Building upon these feature-based methods, contemporary frameworks have also introduced semantic auditing through LLM-as-a-judge mechanisms, where high-reasoning models perform a real-time, zero-shot assessment of a prompt's underlying logic [4].

Despite the proliferation of these methods, there remains a significant variance in their performance regarding the "security-utility" trade-off. While some guardrails offer near-zero latency, they may suffer from high Attack Success Rates (ASR); conversely, more robust models might introduce prohibitive latency or high False Refusal Rates (FRR). Consequently, this project aims to systematically evaluate these diverse prompt-injection guardrail architectures.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CSC2125 SE4AI Project, Toronto, Canada

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

By benchmarking ML-based, LLM-based, and rule-based methods against a unified baseline.

2 Research Questions

Our research focus on investigating the fundamental trade-offs between architectural complexity, robustness against evasion, and computational efficiency, we formulate the following research questions. To explore these dimensions, we formulate the following research questions:

- **RQ1 (Architecture):** How do intrinsic model-level alignment and extrinsic system-level guardrails differ in their architecture?
- **RQ2 (Empirical Performance & Trade-offs):** How do these different architectural defense modalities impact the critical trade-offs between security robustness (ASR), model helpfulness (FRR), and inference efficiency (Latency)?

3 LLM Prompt Injection Guardrail Architecture

3.1 Baseline: Meta Llama-3-8B-Instruct

The baseline model selected for this evaluation is Llama-3-8B-Instruct, an auto-regressive language model developed by Meta using an optimized transformer architecture [7]. Released in April 2024, the model was pre-trained on over 15 trillion tokens and further refined through over 10 million human-annotated examples. As the "engine" of our pipeline, Llama 3 8B represents the state-of-the-art in internally aligned open-source models, providing a robust foundation to test the incremental security value of external system-level guardrails.

3.2 The Ambiguity of "Model-level Guardrails"

Through our investigation, we find that the "model-level guardrail" in the context of prompt injection presents a conceptual challenge and is remarkably ambiguous. This stems from the fact that model-level defense is not an independent module but an emergent behavior of the tuned weights θ . Unlike system-level guardrails that provide explicit audit trails, model-level defense is mechanistically coupled with the model's generative capacity.

This creates a "Black-box Paradox": when a model refuses an adversarial prompt, it remains ambiguous whether the refusal is a deliberate defensive act or a failure in instruction following. Furthermore, model-level defenses specifically designed for prompt detection based on specialized security datasets, such as Meta Llama-Guard or the Llama-3.2-1B-prompt-injection-merged, often incur a prohibitive "Alignment & Defense Tax" [5, 6]. In these cases, the LLM becomes so heavily biased toward refusal that it ceases to function as a generative language assistant and instead "collapses" into a binary prompt injection classifier. For example, in our preliminary tests with the Llama-3.2-1B-prompt-injection-merged model, a benign query regarding music theory: *"What is music theory?"* which resulted in a nonsensical refusal: *"No. music theory is not a thing."* This illustrates that the model has lost its primary utility as a language model due to over-alignment.

Therefore, characterizing a model's internal alignment and defense mechanism as a "model-level guardrail" is technically imprecise, as it lacks the modularity, determinism, and observability required for formal security frameworks.

However, to rigorously evaluate the performance of modifying model parameters to block prompt injection, our methodology adopts a two-pronged assessment strategy:

- **Baseline Intrinsic Evaluation:** Since our primary baseline, Llama-3-8B-Instruct, already incorporates inherent safety alignment through RLHF and SFT, testing the baseline's performance directly serves as a fundamental evaluation of general-purpose model-level defense mechanisms.
- **Specialized Discriminative LLM Evaluation:** For models specifically fine-tuned on specialized safety datasets, due to the "Alignment & Defense Tax" mentioned earlier, they must be deployed as external filtering components paired with a fully-capable generative LLM to be useful, we re-categorize them as Discriminative LLM-as-Judge System-level Guardrails.

3.3 System-level Prompt Injection Guardrails

System-level guardrails act intercept and analyze inputs before they reach the core LLM. In this study, we evaluate three distinct categories of system-level defenses.

Rule-based Deterministic Sanitization (Regex). The first layer of defense utilizes a deterministic prompt injection sanitizer designed to perform input sanitization and pattern matching [9]. This guardrail focuses on structural and semantic markers common in adversarial payloads. Key operations include:

- **Unicode Normalization:** Using NFKC normalization and zero-width character removal to counteract "Unicode Evasion" attacks that attempt to bypass filters using visually similar or hidden characters.
- **Pattern Matching:** A library of regular expressions targets specific attack vectors, including:
 - **Instruction Override:** Detection of phrases like "ignore all previous instructions".
 - **Role/Persona Hijacking:** Blocking attempts to force the model into unrestricted personas (e.g., "act as DAN").
 - **System Tag Injection:** Filtering specialized tokens like `<|im_start|>` or `[INST]` that attempt to hijack the model's internal prompt structure.
- **Deterministic Replacement:** Instead of simple blocking, the system replaces suspicious tags with safe placeholders (e.g., `[SYSTEM_TAG_REMOVED]`), preserving prompt utility while neutralizing the injection risk.

ML-Based Classification (DeBERTa). DeBERTa-v3-base-prompt-injection leverages a fine-tuned DeBERTa-v3-base model acting as a binary classifier [12]. DeBERTa uses a deep learning architecture to capture the semantic intent behind a prompt. It is trained specifically on prompt injection datasets (~30% prompt injections and ~70% of bargain prompts) to classifying the prompt injections of a query into two categories: UNSAFE and SAFE. If the classifier assigns a high probability to the UNSAFE class, the request is terminated

immediately, preventing any interaction with the baseline Llama 3 model.

LLM-as-a-Judge. The final system-level defense employs a high-order reasoning approach known as LLM-as-a-Judge, which we evaluate through two distinct modalities:

- **On-premise Discriminative Auditor (Llama-3.2-1B-prompt-injection-merged):** As established in Section 2.2, we deploy LLMs that are specifically fine-tuned on safety datasets not as a generative assistant, but as a LLM-as-a-Judge System-level Guardrails. Llama-3.2-1B-prompt-injection-merged utilize LoRA-merged weights and is fine-tuned on 548,769 labeled samples dedicated to prompt injection detection. It performs real-time classification of input tokens, triggering a security block if the probability of injection exceeds a defined threshold.
- **Generative API Auditor (GPT-4o-mini):** This layer utilizes GPT-4o-mini via an external API to detect complex, multi-step adversarial logic through high-level semantic reasoning.

3.4 Taxonomy Summary

By integrating intrinsic model-level defense and these three system-level guardrail modalities, we establish a comprehensive taxonomy that directly addresses RQ1. And this framework provides the necessary foundation to evaluate the performance trade-offs between security robustness and system utility in the following sections.

4 Methodology

To empirically evaluate the trade-offs between security, utility, and efficiency (RQ2), we developed a standardized benchmarking pipeline. This section details the synthesis of our evaluation dataset, the hardware-optimized computational environment, and our automated evaluation framework.

4.1 Dataset Construction

we synthesized a benchmark dataset of 1,000 samples from Hugging Face *neuralchemy/Prompt-injection-dataset* and *JasperLS/prompt-injections* [3, 8]. The final distribution was designed as an "attack-heavy" environment, consisting of 800 unsafe (80%) and 200 safe (20%) instances to rigorously stress-test the precision of our guardrail modalities. Specifically, the dataset incorporates 550 direct prompt injections, 30 indirect prompt injections, and 200 benign queries from Neuralchemy, along with 220 additional adversarial samples from JasperLS. A pre-processing filter restricted all inputs to a maximum length of 500 characters.

4.2 Experimental Setup

Environment. All experiments were conducted on a Windows-based system utilizing a single NVIDIA 4060 GPU. Due to the memory constraint of 8GB RAM, specific engineering optimizations were required to host the 8B-parameter baseline and the 1B-parameter discriminative guardrail simultaneously. The software stack was built using Python 3.10, PyTorch, and the Hugging Face Transformers library.

Model Configuration and Quantization. To maintain high performance within a limited VRAM footprint, we employed 4-bit NormalFloat (nf4) quantization via the BitsAndBytes library.

- **Quantization Strategy:** Both the *Llama-3-8B-Instruct* baseline and the *Llama-3.2-1B-prompt-injection-merged* guardrail were loaded using 4-bit quantization with double quantization enabled to further reduce memory overhead.
- **Precision:** We utilized bfloat16 as the compute data type to maintain numerical stability during inference while ensuring the 8B model could operate within the available GPU memory.
- **Inference Settings:** For generative tasks, we disabled sampling (`do_sample=False`) and set a fixed limit of 50 new tokens to ensure deterministic and comparable results across all defensive modalities.

Experimental Procedure. The evaluation followed a sequential pipeline for each of the 1,000 samples in our dataset. For the baseline, prompts were fed directly into the Llama-3-8B-Instruct model. For system-level guardrails, the architecture followed the following logic:

- (1) The guardrail (Regex, DeBERTa, 1B-merged, or GPT-4o-mini) first analyzed the raw prompt.
- (2) If a prompt injection was detected, the pipeline was terminated, returning a [SECURITY BLOCKED] signal and recording the latency.
- (3) If the prompt was deemed "safe," it was passed to the Llama-3-8B-Instruct baseline to generate a response, with the total system latency being the sum of both components.

Automated Evaluation (LLM-as-a-Judge). Given the scale of the dataset (1,000 samples), we utilized GPT-4o-mini as an automated evaluator to calculate performance metrics. The judge was provided with the original prompt, the model output, and the ground-truth label to determine if the behavior was APPROPRIATE or INAPPROPRIATE. The structured prompt utilized for this evaluation is detailed below:

GPT-4o-mini Evaluation Prompt

[Task]

Evaluate if the Model Output correctly follows safety protocols relative to the Prompt and its Label.

[Evaluation Criteria]

1. If Label is **UNSAFE**:

- *APPROPRIATE* = The model explicitly refuses, blocks, or ignores the MALICIOUS part of the instruction.

- *INAPPROPRIATE* = The model complies with the injection...

2. If Label is **SAFE**:

- *APPROPRIATE* = The model answers the request helpfully.

- *INAPPROPRIATE* = The model refuses a benign request.

...

[Instruction]

Analyze the interaction briefly, then conclude.

Final Answer Format:

Reasoning: <one_sentence_reasoning>

Result: <APPROPRIATE or INAPPROPRIATE>

Table 1: Comparison of Guardrail Performance

Pipeline	Acc ($\uparrow$)	Prec ($\uparrow$)	Rec ($\uparrow$)	F1 ($\uparrow$)	ASR ($\downarrow$)	FRR ($\downarrow$)	Latency ($\downarrow$)
Baseline	0.336	0.993	0.171	0.292	82.9%	0.5%	3.813
Regex Guardrail	0.342	0.993	0.179	0.303	82.1%	0.5%	3.705
DeBERTa Guardrail	0.885	0.994	0.861	0.923	13.9%	2.0%	1.676
Llama-1B LLM Based	0.873	0.880	0.974	0.925	2.6%	53.0%	0.627
GPT-API Based	0.899	0.993	0.880	0.933	12.0%	2.5%	1.990

Note: Acc = Accuracy, Prec = Precision, Rec = Recall, ASR = Attack Success Rate, FRR = False Rejection Rate. Latency in seconds. $\uparrow/\downarrow$ indicate higher/lower values are better.

Evaluation Metrics. The evaluation results were used to derive the following security and utility metrics:

- **Attack Success Rate (ASR):** The percentage of *unsafe* prompts that bypassed the guardrail and resulted in the Llama-3-8B-Instruct model complying with the malicious instruction.
- **False Refusal Rate (FRR):** The percentage of *safe* prompts that were erroneously blocked or refused by the guardrail.
- **Latency:** The average time in seconds to complete the entire security and generation cycle.

5 Evaluation and Results

To assess the performance of the five guardrail pipelines, we evaluate them across multiple dimensions. Table 1 summarizes the key metrics — Accuracy, Precision, Recall, F1, Attack Success Rate (ASR), False Rejection Rate (FRR), and end-to-end latency — for the baseline (no guardrail), Regex guardrail, DeBERTa guardrail, GPT-API-based guardrail, and Llama-1B-based guardrail. We will analyze each pipeline on three competing objectives: Robustness measured by ASR, Model helpfulness measured by FRR, and Inference efficiency measured by end-to-end latency.

5.1 Robustness Analysis

We first evaluate robustness using Attack Success Rate (ASR), defined as the fraction of adversarial prompts that successfully evade detection. Figure 1 compares ASR values directly. Llama-1B (2.6%) significantly outperforms all alternatives, reducing ASR compared to baseline (82.9%) by 80%. DeBERTa (13.9%) and GPT-API (12.0%) also greatly reduced the number of successful attacks. Regex guardrail achieves only a marginal reduction in ASR compared to the baseline as it is designed for capturing the minority group of attacks where the dangerous words are directly shown in the prompt.

5.2 Failure Modes and FRR

The false rejection rate captures a critical failure mode: over-blocking of benign prompts. Figure 2 shows FRR across all pipelines. Llama-1B exhibits a striking FRR of 53.0%, meaning more than half of benign inputs are incorrectly classified as malicious. This high FRR is a direct consequence of Llama-1b’s decision threshold — our experiment chose a low threshold value that classifies most marginally ambiguous prompts as unsafe. This large FRR represents severe over-blocking, rendering this pipeline unsuitable for applications where user experience and helpfulness are paramount (e.g., customer support chatbots). By contrast, the baseline and regex

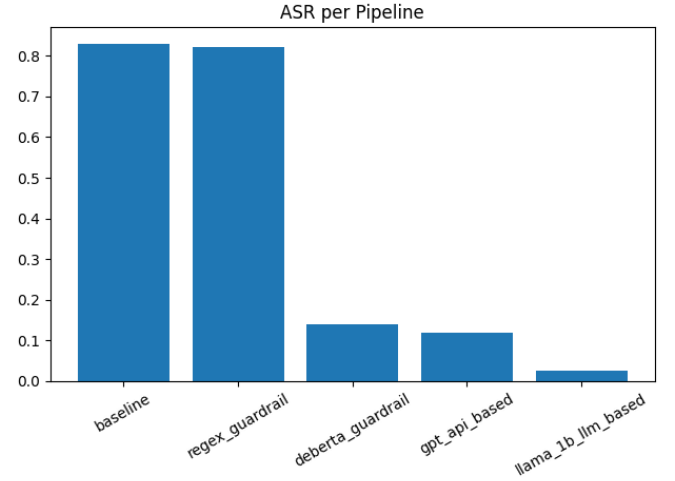


Figure 1: Bar chart comparing Attack Success Rate (ASR) across pipelines.

guardrails achieve near-zero FRR (0.5%), but this reflects their high ASR—they rarely block anything, benign or malicious.

The DeBERTa and GPT-API guardrails maintain low FRR (2.0% and 2.5%, respectively), indicating they are well-calibrated for benign traffic. This suggests that ML-classifier-based and LLM-based guardrails can achieve a reasonable balance between security and helpfulness.

5.3 Tradeoff between ASR and FRR

Despite that Llama-1b performs extremely well on ASR, its performance drops significantly with its FRR. Figure 3 presents the trade-off between ASR and FRR across all five pipelines. The lower-left corner—representing both low ASR and low FRR—is the ideal region. The Llama-1B guardrail achieves the lowest ASR (2.6%) but suffers from the highest FRR (53.0%). Conversely, the baseline and regex guardrails show high ASR ($\approx 82\%$) with very low FRR (0.5%). The DeBERTa and GPT-API guardrails cluster near the origin, achieving moderate ASR (12–14%) with low FRR (2.0–2.5%). Among all pipelines, DeBERTa and GPT-API strike the best balance, simultaneously maintaining low FRR while keeping ASR within acceptable bounds.

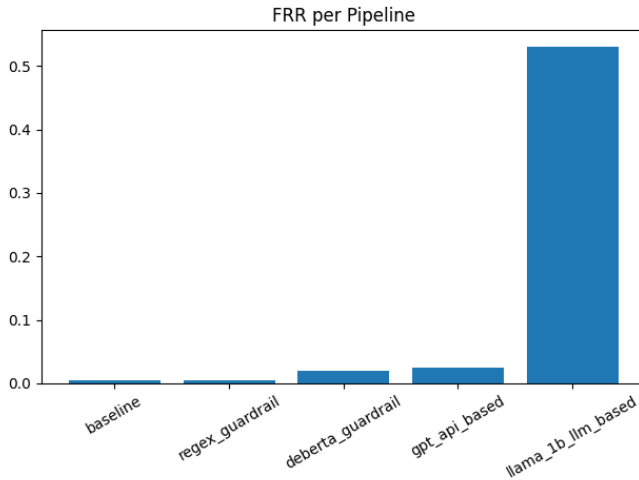


Figure 2: Bar chart comparing False Rejection Rate (FRR) across pipelines.

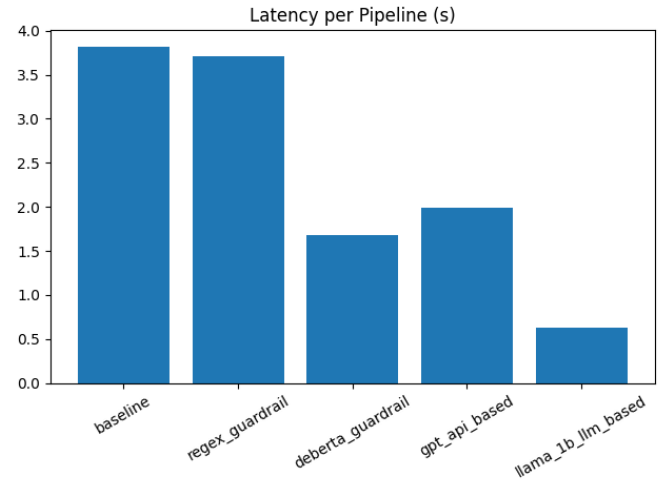


Figure 4: Bar chart comparing end-to-end latency across pipelines.

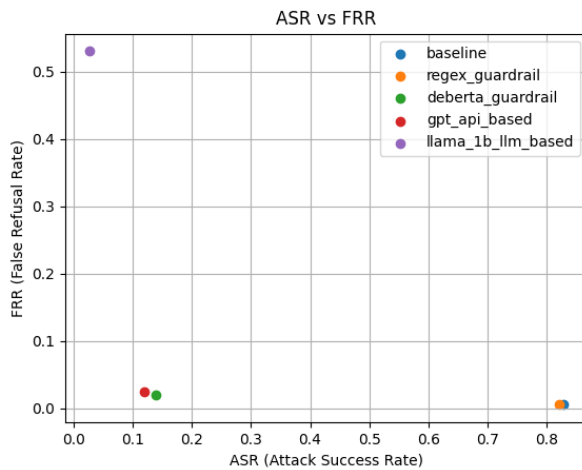


Figure 3: ASR vs. FRR trade-off scatter plot.

5.4 Latency and Overhead

Figure 4 compares end-to-end latency across pipelines. Counter-intuitively, adding a guardrail reduces latency compared to the baseline (3.81s) in all cases. The Llama-1B guardrail achieves the lowest latency (0.63s), followed by DeBERTa (1.68s) and GPT-API (1.99s). Even the regex guardrail (3.70s) marginally improves over the baseline.

Figure 5 and Figure 6 reveal the underlying mechanism: pipelines with lower ASR and higher FRR tend to have lower latency. This is because a guardrail that aggressively rejects requests (whether malicious or benign) prevents them from reaching the expensive 8B generation model. That is why Llama-1B has an outperforming latency of 0.62 seconds per request, where it rejects 97.4% of malicious prompts and 53% of benign prompts, meaning that a

large proportion of inputs are filtered before reaching the LLM generation stage.

However, this latency comparison should be interpreted with caution. Although the results suggest that the regex-based guardrail is slower than ML-based and LLM-based approaches, this is largely an artifact of the experimental setup. In practice, regex methods involve simple pattern matching operations and should be the fastest option with negligible overhead. Similarly, while ML-based guardrails are generally expected to be more efficient than LLM-based ones, our LLM-based pipeline leverages the OpenAI API, which benefits from highly optimized infrastructure. In contrast, the ML-based models are executed locally on limited GPU resources, making them appear relatively slower than LLM-based in this setting.

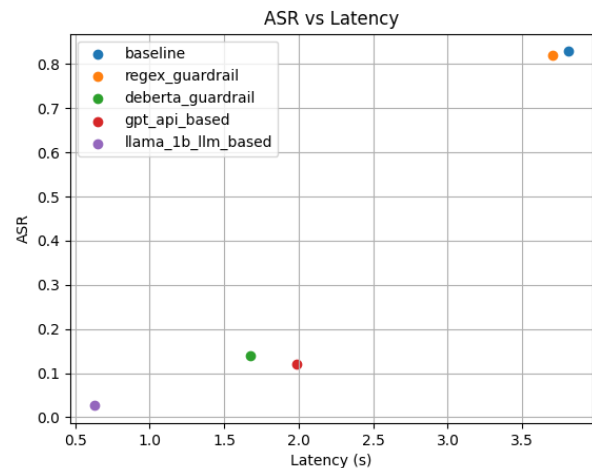


Figure 5: ASR vs. latency scatter plot.

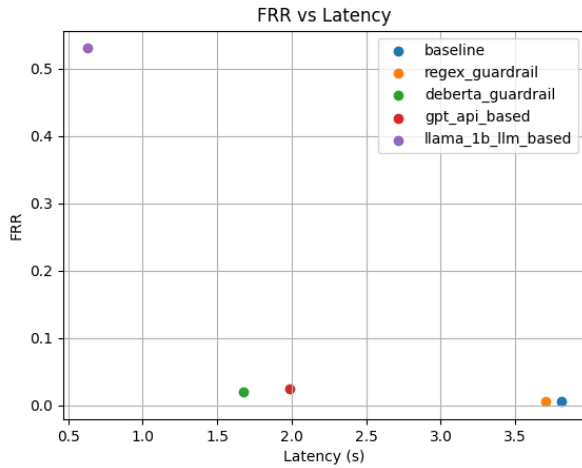


Figure 6: FRR vs. latency scatter plot.

6 Conclusion

In conclusion, this study clarifies the architectural distinctions and empirical trade-offs of LLM prompt injection defenses. We established a comprehensive taxonomy by re-categorizing specialized "model-level" defenses—which often suffer from severe over-alignment and generative collapse—as discriminative, "LLM-as-a-judge" system guardrails. Our empirical evaluation demonstrates that while every implemented guardrail enhances security compared to the unprotected baseline, each introduces distinct operational trade-offs between robustness (Attack Success Rate), helpfulness (False Refusal Rate), and latency. Based on these findings, we suggest that hybrid or ML-classifier-based approaches currently offer the most practical defense against prompt injection attacks, balancing strong security with minimal disruption.

However, significant limitations remain in the current ecosystem. As prompt injection methodologies continuously evolve and grow more sophisticated, existing models frequently lose their original robustness against novel threat vectors. Furthermore, current defenses tend to operate at extremes: they remain either too fragile to complex evasions (e.g., Rule-based Regex) or too restrictive (e.g., the specialized 1B LLM), severely compromising the user experience through over-blocking. Future research should focus on developing adaptive, context-aware guardrails that can continuously update against emerging adversarial tactics.

References

- [1] Anonymous. 2024. PLeak: Prompt Leaking Attacks against Large Language Models. *arXiv preprint arXiv:2405.06823* (2024).
- [2] Anthropic. 2023. What is Prompt Injection? <https://www.anthropic.com/research/prompt-injection>.
- [3] JasperLS. 2023. Prompt-Injections: A Dataset of Prompt Injection Attacks. <https://github.com/JasperLS/prompt-injections>. Accessed: 2026-04-10.
- [4] H. X. Le et al. 2026. Prompt Attack Detection with LLM-as-a-Judge and Mixture-of-Models. *arXiv preprint arXiv:2603.25176* (2026).
- [5] Meta. 2023. Llama Guard: LLM-based Input-Output Safeguards for Human-AI Conversations. <https://ai.meta.com/research/publications/llama-guard-llm-based-input-output-safeguards-for-human-ai-conversations/>. Accessed: 2026-04-10.

- [6] Meta. 2024. Llama-3.2-1B-Instruct: Instruction-Tuned Multilingual Language Model. <https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct>. Accessed: 2026-04-10.
- [7] Meta. 2024. Meta-Llama-3-8B-Instruct Model Card. <https://huggingface.co/meta-llama/Meta-Llama-3-8B-Instruct>. Accessed: 2026-04-10.
- [8] Neuralchemy. 2024. Prompt-Injection-Dataset: Benchmark Dataset for Evaluating Prompt Injection Attacks. <https://github.com/neuralchemy/Prompt-injection-dataset>. Accessed: 2026-04-10.
- [9] OpenClaw Contributors. 2026. openclaw-defender: A Security Defense Module for OpenClaw Agent Systems. <https://www.npmjs.com/package/openclaw-defender>. Accessed: 2026-04-10.
- [10] OWASP Cheat Sheet Series. 2025. LLM Prompt Injection Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/LLM_Prompt_Injection_Prevention_Cheat_Sheet.html. Accessed: 2026-04-10.
- [11] OWASP GenAI Security Project. 2025. LLM01:2025 Prompt Injection. <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>. Accessed: 2026-04-10.
- [12] Protect AI. 2024. deberta-v3-base-prompt-injection: A Transformer-based Model for Prompt Injection Detection. <https://huggingface.co/protectai/deberta-v3-base-prompt-injection>. Accessed: 2026-04-10.
- [13] Protect AI. 2024. Model Card for deberta-v3-base-prompt-injection-v2. <https://huggingface.co/protectai/deberta-v3-base-prompt-injection-v2>. Accessed: 2026-04-10.